國立臺灣科技大學
電 子 工 程 系

碩士學位論文

應用於 nFAPI 分離架構之吞吐量—延遲權衡的自適應時序控制

Adaptive Timing Control for Throughput—Latency Trade-off in nFAPI Split

研 究 生：徐銘鴻

學　　號：M11302209

指導教授：鄭瑞光博士

中華民國一百一十五年六月一十五日

Recommendation Letter

M11302209

碩士學位論文指導教授推薦書
Master's Thesis Recommendation Form

系所：
Department/Graduate Institute: 電子工程系 Department of Electronic and Computer Engineering

姓名：
Name: 徐銘鴻 MING HONG HSU

論文題目：
(Thesis Title): 應用於 nFAPI 分離架構之吞吐量－延遲權衡的自適應時序控制
Adaptive Timing Control for Throughput–Latency Trade-off in nFAPI Split

係由本人指導撰述，同意提付審查。
This is to certify that the thesis submitted by the student named above, has been written under my supervision. I hereby approve this thesis to be applied for examination.

指導教授簽章：
Advisor's Signature _____

共同指導教授簽章（如有）：
Co-advisor's Signature (if any) _____

日期：
Date(yyyy/mm/dd) _____/_____/_____

Approval Letter

M11302209

碩士學位考試委員審定書

Qualification Form by Master's Degree Examination Committee

系所： Department/Graduate Institute	電子工程系 Department of Electronic and Computer Engineering
姓名： Name	徐銘鴻 MING HONG HSU
論文題目： (Thesis Title)	應用於 nFAPI 分離架構之吞吐量－延遲權衡的自適應時序控制 Adaptive Timing Control for Throughput–Latency Trade-off in nFAPI Split

經本委員會審定通過，特此證明。

This is to certify that the thesis submitted by the student named above, is qualified and approved by the Examination Committee.

學位考試委員會

Degree Examination Committee

委員簽章：

Member's Signatures _____ _____

_____ _____

_____

指導教授簽章：

Advisor's Signature _____

共同指導教授簽章（如有）：

Co-advisor's Signature (if any) _____

系所（學程）主任（所長）簽章：

Department/Study Program/Graduate Institute Chair's Signature _____

日期：

Date(yyyy/mm/dd) _____/_____/_____

摘　要

在開放式無線接取網路（O-RAN）架構中，功能拆分選項 6（Option 6，即標準化之 nFAPI）藉由分離媒體存取控制（MAC）與實體（PHY）層，提供了在一般封包交換網路上部署的彈性，避免了時效性網路（TSN）硬體同步的高昂成本。然而，將 MAC 層遷移至雲端環境中的虛擬化網路功能（VNF），會不可避免地引入不可預期的傳輸與處理延遲。現有的開源實作多採用靜態的提前時槽（slots ahead）配置，無法適應動態的網路抖動，導致嚴重的排程失敗（如遺失時槽封包）與吞吐量下降。爲解決此挑戰，本論文針對非理想的前傳（fronthaul）網路環境，提出了一套自適應時序控制框架。該方法採用指數平滑移動平均（EWMA）演算法動態估算網路延遲，並主動調整排程的提前時槽，確保控制訊息能準確落入實體網路功能（PNF）的嚴格時序視窗內。我們在整合商用 Pegatron O-RAN 無線單元（O-RU）的 OpenAirInterface（OAI）測試平台上進行實驗驗證。結果顯示，本論文提出的自適應機制能有效消除同步失敗，維持穩定的混合自動重傳請求（HARQ）來回延遲（RTT），並在嚴重的網路壅塞下確保系統吞吐量的韌性與穩定。

Abstract

In the Open Radio Access Network (O-RAN) architecture, functional split Option 6 (standardized as nFAPI) provides deployment flexibility over general packet-switched networks by decoupling the Medium Access Control (MAC) and Physical (PHY) layers, thereby avoiding the very high costs of Time-Sensitive Networking (TSN). However, migrating the MAC layer to a Virtualized Network Function (VNF) in a cloud environment introduces non-deterministic transport and processing latency. Existing open-source implementations rely on static slots ahead timing configurations, which fail to adapt to unpredictable network jitter, leading to severe scheduling failures and throughput degradation. To address this challenge, this thesis proposes an adaptive timing control framework tailored for non-ideal fronthaul environments. The proposed method employs an Exponentially Weighted Moving Average (EWMA) to dynamically estimate network delay and actively adjust the scheduling slots ahead, ensuring that control messages arrive within the strict timing window of the Physical Network Function (PNF). Experimental validations conducted on an OpenAirInterface (OAI) testbed integrated with a commercial Pegatron O-RAN Radio Unit (O-RU) demonstrate that the proposed adaptive mechanism effectively eliminates synchronization failures, maintains stable Hybrid Automatic Repeat Request (HARQ) round-trip times (RTT), and ensures resilient system throughput under severe network congestion.

Acknowledgements

I would like to express my deepest gratitude to my advisor, Professor Ray, for his dedicated guidance and patient instruction throughout the course of my research. His mentorship has been invaluable, allowing me to gain both academic insights and practical experience that greatly enriched my personal growth.

I am also sincerely thankful to my parents, whose unwavering support enabled me to pursue my master's degree without worry and focus wholeheartedly on my studies.

I would like to thank my friend—Tzu-Yu, Mei-Chun, Yu-Hsuan, Chih-Hsuan, and Ching-Han—as well as the group "Ba Gwai." During times of stress, they were always there to listen, have fun, and help me unwind, serving as my warmest emotional outlet. My gratitude also goes to my badminton group and NTUT basketball friends, who consistently encouraged me to exercise whenever I felt lazy—otherwise, I surely would have gained much more weight.

In the laboratory, I was fortunate to join a team with a harmonious and collaborative atmosphere. The senior members generously shared their knowledge and guided us with care, while the junior members were always willing to help. I am especially grateful for the tremendous support during my oral exam preparation. Beyond academics, we also enjoyed many group activities together, such as Christmas gift exchanges and dinners in Gongguan, which not only strengthened our bonds but also filled my graduate life with joy.

I am also very thankful for the group of wonderful friends I made in the lab—Big Winnie, Joanna, JOJO, YH, ZX, Heidi, Kenny, Ming, Tobby, Richard, Mira, Will, Little Winnie, and Nino. Thank you for always being there to chat, share daily life, and accompany me through most of my master's journey.

Lastly, I wish to thank my classmates Jerry, Joe, Wilfrid, Johnson, Sheryl, and Sylvia. From preparing midterm exams in the first year to oral exams in the second year, the mutual support and friendship we shared have become some of my most cherished memories.

Contents

List of Figures

List of Tables

Chapter 1 Introduction

1.1 Research Background and O-RAN Architecture Evolution

In the architectural evolution of the fifth-generation (5G) and the upcoming sixth-generation (6G) mobile communication standards, the Radio Access Network (RAN) is undergoing a fundamental transformation from the traditional monolithic base station to a highly virtualized, cloudified, and disaggregated architecture. Under the traditional architecture, baseband processing and Radio Frequency (RF) functions were tightly bound to the proprietary hardware equipment of a single vendor, which not only limited the flexibility of network deployment but also significantly increased the capital and operational expenditures of operators.

With the rise of Cloud-RAN (C-RAN), disaggregated deployment methods have gradually become popular, centralizing the Baseband Processing Unit (BBU) into a remote BBU Pool while leaving the Remote Radio Head (RRH) at the edge site, connecting the two via the Common Public Radio Interface (CPRI). However, the C-RAN architecture is still limited by the massive fronthaul bandwidth requirements and strict latency constraints. To overcome these bottlenecks and completely break the dilemma of vendor lock-in, the O-RAN Alliance [1] and the 3rd Generation Partnership Project (3GPP) jointly promoted further disaggregation of the RAN, formally splitting the traditional BBU and RRH logically and physically into three independent network entities: the Central Unit (CU), Distributed Unit

(DU), and Radio Unit (RU) [2,3].

In this disaggregated model, ”Functional Split Options” have become the most critical design core for determining network performance, hardware complexity, bandwidth consumption, and latency tolerance [4,5]. 3GPP has standardized a variety of functional split options to adapt to diverse network requirements and deployment scenarios.

1.1.1 Split Option 2

Split Option 2 is a High Layer Split (HLS) architecture that primarily separates the CU and DU. Under this split, higher-layer protocols such as the Packet Data Convergence Protocol (PDCP) and Radio Resource Control (RRC) run centrally in the CU, while lower-layer Radio Link Control (RLC), Medium Access Control (MAC), and Physical Layer (PHY) remain in the DU. This architecture has relatively loose fronthaul bandwidth and latency requirements, making it easy to deploy over general packet-switched networks. The design of its synchronization mechanism is also very simple; due to its high tolerance, it only relies on flow control and deep buffering to operate stably [4], and is widely recognized as the standard configuration with the best cost-effectiveness ratio, especially suitable for non-real-time data transmission scenarios.

1.1.2 Split Option 6

Split Option 6, also known as the MAC-PHY split, precisely places the architectural separation point between the MAC layer and the PHY layer.

In this option, the DU is responsible for processing upper-layer protocols including the MAC layer, while the RU retains complete physical layer (PHY) processing capabilities. Since the fronthaul network transmits MAC Protocol Data Units (MAC PDUs) scheduled by the MAC layer, its bandwidth requirements are dynamically adjusted based on the actual load of the users. Notably, although the nFAPI-based Split Option 6 architecture also supports deployment over general packet-switched networks, its synchronization requirements are completely different from Split 2. It must strictly follow the scheduler's synchronization baseline at every slot level, ensuring data is delivered precisely on time from the VNF to the PNF. During this process, the operating system scheduling delays on the VNF side, as well as the unknown transmission delays and jitter introduced by the packet-switched network, are major challenges that must be overcome to maintain precise time synchronization [6].

1.1.3 Split Option 7.2

Split Option 7.2x (Low-PHY split) places the split point inside the physical layer, and is currently the primary Lower Layer Split (LLS) standard promoted by the O-RAN Alliance [7]. Although moving partial computations (such as FFT/IFFT) to the RU alleviates fronthaul pressure, its fronthaul transmits frequency-domain IQ symbols processed by the physical layer, demanding extremely high bandwidth. Because of this, Split 7.2 must be deployed under very strict network conditions, and its reliance on hardware equipment (such as hardware-level Precision Time Protocol, PTP) results in extremely high deployment costs [8]. However, precisely

because it has a strict proprietary network environment and transmits a stable, continuous stream of IQ data, its synchronization mechanism is relatively stable and does not require special handling for latency fluctuation issues caused by different offered loads.

1.1.4 Split Option 8

Split Option 8 is the most traditional lower-layer split architecture, placing the split point between the Radio Frequency (RF) and the Physical Layer (PHY). Under this architecture, all baseband processing functions (including complete PHY, MAC, and higher-layer protocols) are centralized in the DU or CU, while the RU is only responsible for RF signal transmission and reception. This configuration produces a fronthaul data stream dominated by time-domain IQ samples, usually relying on CPRI or enhanced CPRI (eCPRI) for transmission. Although it highly centralizes hardware resources, its fronthaul network bandwidth requirements are extremely high, and it is strictly constrained by system bandwidth and antenna count. Furthermore, its latency limits are extremely strict (maximum allowable one-way latency is $250\mu s$ [2]), thus, similar to Split 7.2, it heavily relies on high-quality proprietary optical networks and precise timing synchronization hardware.

1.2 Network Architecture and Challenges Based on Split Option 6

This thesis focuses on the architectural design and application of Split Option 6. As mentioned earlier, while adopting lower-layer splits (such as Split Option 7.2) can further centralize computational resources, it requires a more expensive PTP deployment environment and a transmission network with extremely high demands for low latency, which will lead to high capital and operational expenditures. In light of this, choosing the Split Option 6 approach, which leaves the complete physical layer (Monolithic PHY) at the remote node, can strike the best balance between resource requirements and cost-effectiveness. According to research by Khan et al. [9], cost-benefit trade-off analysis for 5G NR small cells indicates that compared to strict lower-layer splits, the architecture separating the MAC and PHY layers can significantly reduce the required fronthaul bandwidth while ensuring goodput, thereby achieving the goal of lowering overall deployment costs. Therefore, in non-ideal or resource-constrained network environments, Split Option 6 becomes the ideal choice that balances performance and cost.

1.2.1 Introduction to the FAPI Interface

Within the internal architecture of 5G base stations, communication between the MAC layer and PHY layer relies on standardized interfaces. The Functional Application Platform Interface (FAPI) introduced by the Small Cell Forum (SCF) was created for this purpose [10]. FAPI provides a stan-

dardized format for control messages and data payloads, allowing MAC and PHY components from different vendors to interoperate. However, the traditional FAPI design assumes that the MAC and PHY are located within the same physical device, and the two exchange data quickly with low latency via shared memory, which limits the geographical disaggregation and distributed deployment of base station functions.

1.2.2 Introduction to the nFAPI Interface

To overcome the physical deployment limitations of FAPI and realize the Split Option 6 architecture across networks, SCF further developed the network Functional Application Platform Interface (nFAPI) [11]. The core innovation of nFAPI lies in encapsulating FAPI messages, which originally relied on shared memory, into a network packet protocol that can be transmitted over Ethernet or IP networks [12]. This technological breakthrough grants Virtual Network Functions (VNFs) immense flexibility, allowing cloudified MAC functions running on Commercial Off-The-Shelf (COTS) servers to remotely control physical PHY devices on Physical Network Functions (PNFs) over packet-switched networks.

It is particularly important to note that according to 3GPP TR 38.801 [2] specifications, the ideal fronthaul one-way latency limit for Split Option 6 is $250\mu s$. However, the SCF provides a more practical, special interpretation for nFAPI's latency tolerance in its specifications [11,13]. The SCF indicates that nFAPI is designed for packet-switched IP networks, and its fronthaul latency budget is primarily determined by the configuration of the autonomous HARQ retransmission mechanism. By configuring two

autonomous HARQ retransmissions, nFAPI can tolerate up to 6 ms of fronthaul delay (including frame delay and delay jitter) [13]. Furthermore, in a non-ideal fronthaul environment, by fine-tuning the HARQ parameters of the S-DU (such as K0, K1, K2) and the RACH timer, combined with the built-in delay management mechanism of the nFAPI P7 interface and the SCTP protocol adapted for the P5 interface, nFAPI can even support highly delayed and highly jittered transmission conditions of up to 30 ms [11]. This design specifically for non-ideal networks makes Split Option 6 highly practical and flexible in deployment.

1.2.3 Comprehensive Comparison of Split Technologies

To more intuitively present the differences between different split options, Table 1.1 provides a comprehensive comparison of Split 2, Split Option 6, Split 7.2x, and Split 8 in terms of protocol boundaries, bandwidth requirements, and latency limits.

This thesis focuses on the deployment and optimization of nFAPI because it provides excellent network flexibility, allowing network operators to logically split the base station and deploy it on two independent machines, even connected via general packet-switched networks. However, this physical split also brings immense challenges: in a non-deterministic network environment, how to maintain time synchronization between the two nodes and conduct precise delay management has become the primary difficulty in ensuring stable system operation.

Table 1.1: 5G RAN Split Deployment Comparison Table

Split Option	**Split 2**	**Split 6 (nFAPI)**	**Split 7.2x**	**Split 8**
Protocol Boundary	PDCP / RLC	MAC / PHY	Low-PHY / High-PHY	RF / PHY
Fronthaul Data Type	PDCP PDU	MAC PDU (Packetized)	Frequency-domain IQ symbols	Time-domain IQ samples
Standardized Interface	F1	SCF nFAPI	O-RAN WG4 eCPRI	CPRI / eCPRI
Bandwidth Driver	Actual user traffic	Actual user traffic	Bandwidth, spatial layers	System bandwidth, antennas
Latency Limit [2,13]	$1.5 \sim 10ms$	$250\mu s$ (3GPP) $6 \sim 30ms$ (SCF)	$250\mu s$	$250\mu s$
Optimal Deployment	Non-real-time services	Small cells, indoor private networks	Macro cells	C-RAN, dedicated fiber

1.3 Research Motivation and Problem Definition

Although Split Option 6 has flexible timing adjustment capabilities and better tolerance for non-ideal fronthaul networks, it still faces severe challenges in practical deployment. As radio access networks move towards virtualization and cloudification, the hosted MAC layer functions are typically deployed as Virtual Network Functions (VNFs) on general-purpose processors. However, in this general-purpose computing environment, non-deterministic operating system scheduling jitter and processing delay uncertainty often impact timing-sensitive baseband processing [6,14].

Specifically, in existing open-source implementations (such as OpenAirInterface), the timing control of the nFAPI interface mostly relies on a statically configured ”slots ahead”. This hardcoded approach cannot dynamically adapt to the rapidly changing transmission delays and network jitter in packet-switched networks. When bursty congestion occurs in the fronthaul network or latency spikes appear in the VNF computing environment, scheduling commands and control messages are highly likely to miss their scheduled timing deadlines, thereby generating Missing Slot Packet errors. These types of errors not only lead to communication interruptions but also limit the VNF’s elastic scalability in dynamic cloud resource environments.

Therefore, the core research question of this thesis is defined as follows: In a cloud environment with non-deterministic processing loads and

network jitter, how to dynamically and accurately compensate for end-to-end latency in the nFAPI Split Option 6 architecture to ensure the stability of timing synchronization?

1.4 Related Works

In recent years, the rapid development of the Open Radio Access Network (O-RAN) architecture has prompted many studies to optimize the performance and network latency management of the Split Architecture. For instance, the X5G testbed proposed by Villa et al. [8] demonstrated an advanced 5G O-RAN deployment solution based on hardware acceleration. This research successfully achieved commercial-grade peak throughput by offloading the massive computations of the Physical Layer (PHY) to dedicated NVIDIA GPUs and Mellanox SmartNICs. However, this architecture, which pursues ultimate performance, highly relies on expensive proprietary hardware equipment, and its prerequisite is that it must operate in an ideal network environment equipped with hardware-level Precision Time Protocol (PTP) synchronization. These strict conditions greatly limit its applicability and deployment flexibility in general packet-switched networks filled with unpredictable latency and jitter.

To address the limitations of high hardware costs and strict network requirements mentioned above, this thesis proposes a highly universal software solution for non-ideal fronthaul environments. Unlike X5G, which relies on hardware acceleration to overcome latency issues, and unlike traditional methods that rely on static configurations to compromise on delay jitter, this thesis designs an ”Adaptive Slots Ahead Control Mechanism”

based on Exponentially Weighted Moving Average (EWMA) delay management specifically for the nFAPI architecture. This mechanism can run on general COTS servers, dynamically tracking and filtering network jitter, and actively adapting to unpredictable network congestion. Experimental results show that the method proposed in this thesis can effectively stabilize the HARQ loop latency (RTT) and maintain system throughput without relying on high-end proprietary hardware or precise clock synchronization equipment, providing a more cost-effective and flexible alternative for O-RAN deployment.

Simultaneously, current academic research on O-RAN delay management can be divided into three levels: planning, management, and scheduling. Although existing literature [15–18] has proposed optimization schemes at various levels, most focus on long-term orchestration or specific hardware acceleration. This thesis fills the gap in addressing the microsecond-level phase synchronization and latency challenges inherent in Split Option 6, presenting a unique academic contribution to realizing dynamic time synchronization under a software-defined network architecture.

1.5 Thesis Contributions

In response to the aforementioned issues, the main contributions of this thesis are as follows:

- **Multi-dimensional Latency Quantitative Analysis for nFAPI Split Option 6:** We systematically analyzed and quantified the non-deterministic factors affecting synchronization failure, including Linux kernel schedul-

ing jitter, VNF processing load, and network queuing delays.

- **Adaptive Slots Ahead Mechanism:** We proposed a dynamic adjustment mechanism based on Node Sync and Delay Management. It uses an EWMA model to smooth timing information to estimate real-time jitter trends, and dynamically adjusts the slots ahead (S_{ahead}) to ensure the punctuality of scheduling messages.

- **End-to-End Commercial Testbed Verification:** We implemented the proposed mechanism on the OpenAirInterface platform and combined it with a commercial UE and O-RU for verification. Experimental results show that this mechanism can effectively eliminate missing slot packet errors and maintain stable network throughput and low latency characteristics in highly congested environments.

Chapter 2 System Architecture

Before delving into system details, this study first introduces the adopted system framework. To integrate the network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the system proposed in this study adopts a resilient two-stage split architecture. The network is first connected through the Split 6 nFAPI interface between network functions, and then converted to the standard O-RAN Split 7.2x interface to connect to a commercial Pegatron O-RU. Figure 2.1 illustrates this system architecture and its delay management mechanism.

2.1 End-to-End Architecture

This system establishes a complete 5G end-to-end communication link extending from the core network to the user equipment:

- **Core Network (CN):** The core component responsible for routing, session management, and user data authentication.
- **O-RAN Central Unit (O-CU):** The logical node handling higher-layer protocol stacks, such as Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects to the distributed unit via the F1 interface.
- **O-RAN Distributed Unit (O-DU):** To optimize computational performance and deployment flexibility, the system divides the O-DU into O-DU High and O-DU Low. O-DU High executes as a Virtual

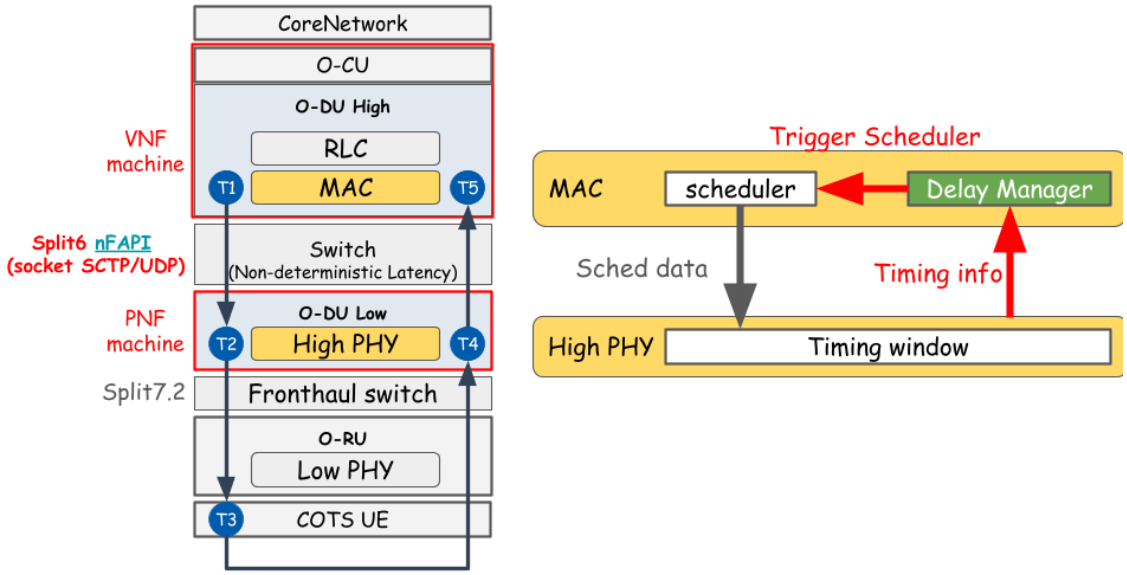


Figure 2.1: System architecture and delay management mechanism.

Network Function (VNF) on virtual machines, while O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.

- **Switch:** A commercial network switch located between the VNF and PNF. This intermediate layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably introduces non-deterministic latency.

- **O-RAN Radio Unit (O-RU):** A dedicated hardware unit responsible for Low PHY processing, Digital Beamforming, and Radio Frequency (RF) transmission.

- **Commercial Off-The-Shelf User Equipment (COTS UE):** Standard commercial smartphones or modems acting as the network's terminal receivers.

2.2 O-DU High: MAC and Delay Manager

In the Medium Access Control (MAC) layer of O-DU High, the core components handling the timing mechanism include the Scheduler and the nFAPI Delay Manager:

- **Scheduler:** Responsible for allocating available radio resources (e.g., time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmission.
- **nFAPI Delay Manager:** Serves as the key control unit of this delay-aware architecture. Due to the unpredictable and unstable delay across the switch between the VNF and PNF, the delay manager continuously receives timing feedback from the lower physical layer. It utilizes this information to dynamically calculate network jitter and actively adjust the **slots ahead (**S_{ahead}**)** to trigger the scheduler. This calculation ensures that the scheduling command (`Sched data`) is generated in advance to compensate for the network delay.

2.3 O-DU Low: High PHY and Timing Window

O-DU Low manages the High PHY baseband processing. At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- **Timing Window:** A predefined and strict time interval. When schedul-

ing messages (especially `DL_TTI.request`) sent by O-DU High traverse the network, they must accurately arrive and fall exactly within this window. Otherwise, the High PHY will reject the message, resulting in the inability to process data for that slot.

- **Timing Information (Δt_{arrive}):** The PNF continuously monitors the exact arrival offset of control messages relative to the transmission deadline. This offset is denoted as Δt_{arrive}, representing the remaining time margin within the timing window. The PNF periodically feeds this information back to the MAC layer delay manager. This establishes a closed-loop feedback control system to compensate for unpredictable transmission delay variations between the VNF and PNF machines.

2.4 Key Verification Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm in non-ideal environments, this study defines the following key verification metrics:

- **MAC Layer HARQ RTT ($T_5 - T_1$):** Specifically measures the Hybrid Automatic Repeat Request (HARQ) loop delay at the MAC layer. This metric precisely reflects the feedback efficiency and operational health of the link layer.

- **VNF-PNF Downlink Delay ($T_2 - T_1$):** Measures the accumulated one-way downlink latency from the virtualized machine (VNF) to

the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact introduced by the intermediate Ethernet switch.

Note: RTT stands for Round-Trip Time. To verify the stability of the split architecture under non-ideal fronthaul conditions, this study simulates network delays using Linux Traffic Control (TC) on the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments.

To validate the proposed performance optimization in a realistic network environment, the system design utilizes a commercial-grade deployment framework:

- **O-RAN Split 7.2x Interoperability.**
  Although the Small Cell Forum (SCF) has standardized the **network Functional Application Platform Interface (nFAPI)** [12] for the MAC-PHY split (Option 6), commercial Radio Units (O-RUs) natively support O-RAN Split Option 7.2x. To achieve seamless integration with open-source protocol stacks like OpenAirInterface [19], this system bridges these interfaces to ensure full-scale commercial interoperability.

- **Hybrid System Architecture for End-to-End Verification.**
  To verify the performance of nFAPI in a fully commercial ecosystem, this paper adopts a **hybrid architecture** (Split Option 6 + Split Option 7.2). In this model, the system provides a Translation/Interworking Layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with the commercial Pegatron O-RU. This setup allows the use of **Commercial Off-The-Shelf (COTS)** user

equipment to verify nFAPI performance in a real-world end-to-end environment.

Chapter 3 Proposed Method

This chapter details the proposed dynamic timing adjustment algorithm. To address the unstable transmission latency during packet transmission between the PNF and VNF, we propose an Adaptive Slots Ahead mechanism.

3.1 Delay Management Architecture

The timing control of this system focuses on the logical layer of synchronization and delay management, ensuring that the slots ahead at the VNF side can flexibly respond to network jitter without causing unexpected oscillations. This architecture operates through two main logical components that interact seamlessly:

1. **Node Sync:** Ensures alignment of frame and slot boundaries between the VNF and PNF. It monitors timing offsets to maintain synchronization, thereby absorbing low-layer transmission jitter.

2. **Delay Management:** Focuses on slot-level delay management. It dynamically analyzes the statistical data reported by the PNF through a convergence optimization algorithm to determine the global target slots ahead (S_{ahead}), ensuring that scheduling commands always comply with the PNF's strict timing window constraints.

3.2 Node Sync

To achieve microsecond-level phase alignment, the system adopts a timestamp exchange mechanism similar to IEEE 1588 (PTP). When the VNF receives the *UL_node_sync* message, it calculates the instantaneous timing offset ($offset$) based on four timestamps (t_1 to t_4). To avoid instantaneous spikes, an Exponentially Weighted Moving Average (EWMA) model is used to smooth the $offset$ and its variation ($error$), generating an average offset (μ_{offset}) and a mean absolute deviation (dev_{offset}), which are adjusted using smoothing factors α and β. Then, a dynamic synchronization threshold ($Threshold$) is derived directly from the estimated jitter (dev_{offset}). If the absolute offset exceeds this threshold, the system decomposes the offset into a slot adjustment (S_{adj}) and a microsecond adjustment (μs_{adj}) based on the slot duration (slot_duration). These parameters are subsequently used by the VNF Timing Actuator to precisely align the local clock. This process is summarized in Algorithm 1.

Algorithm 1 Node Sync

Require: t_1, t_2, t_3 (PNF *UL_node_sync*), t_4 (VNF local reception timestamp)

Ensure: S_{adj} (Slot adjustment), μs_{adj} (Microsecond adjustment)

Notation:

$offset$ (Instantaneous timing offset), μ_{offset} (EWMA mean offset)

dev_{offset} (EWMA mean absolute deviation / jitter)

$Threshold$ (Dynamic synchronization threshold), α, β (Smoothing factors)

slot_duration (Slot duration in μs)

Phase 1: Timestamp Processing and Wraparound Protection

$d_1 \leftarrow t_2 - t_1$

$d_2 \leftarrow t_4 - t_3$

Phase 2: Offset and Dynamic Jitter Estimation

$offset \leftarrow (d_1 - d_2)/2$

$error \leftarrow offset - \mu_{offset}$

$\mu_{offset} \leftarrow \mu_{offset} + \alpha \cdot error$

$dev_{offset} \leftarrow dev_{offset} + \beta(|error| - dev_{offset})$

$Threshold \leftarrow dev_{offset}$

Phase 3: Phase Alignment

if $|offset| > Threshold$ **then**

    $S_{adj} \leftarrow \lfloor offset/\text{slot\_duration} \rfloor$

    $\mu s_{adj} \leftarrow offset \pmod{\text{slot\_duration}}$

    Apply S_{adj} and μs_{adj} to adjust the local clock

else

    State $\leftarrow$ **Synchronized**

    $S_{adj} \leftarrow 0$, $\mu s_{adj} \leftarrow 0$

end if

return $S_{adj}, \mu s_{adj}$

To combat fluctuating network latency and maintain scheduling robustness, this paper proposes a dynamic delay management strategy. This strategy intelligently trades off between throughput and latency by introducing

the concepts of predicted risk and historical risk debt. Upon receiving the worst-case delay offset (Δt_{arrive}) reported by the PNF, the system calculates an Exponentially Weighted Moving Average (EWMA) for the mean delay (μ_{delay}) and jitter deviation (dev_{delay}) using smoothing factors α and β.

Before making tracking adjustments, the system evaluates the potential tail risk ($Risk_{pred}$) based on the maximum of the observed offset and the mean delay, augmented by the jitter deviation. This risk is then accumulated into a history-weighted risk debt ($Debt_{risk}$), which represents a decaying memory of past network instability. If a deadline violation occurs ($\Delta t_{arrive} > 0$) or a high risk is predicted ($Risk_{pred} > 0$), the algorithm takes immediate action to preserve throughput by incrementing the current trigger timing (S_{curr}).

Conversely, to optimize latency during safe operations, the algorithm checks if the historical debt is cleared ($Debt_{risk} < \epsilon$) and if the system has maintained sustained stability over consecutive observations ($count_{stable} \geq N_{stable}$, where N_{stable} scales dynamically). If both conditions are met, the algorithm cautiously decreases the advance time. In scenarios where the risk is low but stability evidence is still accumulating or debt is being cleared, the current timing is maintained. This asymmetric response ensures connection continuity while avoiding ping-pong oscillations. The core adjustment strategy is condensed in Algorithm 2.

Algorithm 2 Delay Management

Require: Δt_{arrive} (Observed timing offset), S_{curr} (Current trigger timing)

Ensure: S_{next} (Trigger timing for the next MAC schedule)

Notation:

μ_{delay} (EWMA mean delay), dev_{delay} (EWMA mean absolute deviation / Jitter)

$Risk_{pred}$ (Predicted timing risk), $Debt_{risk}$ (History-weighted risk debt)

$count_{stable}$ (Consecutive stable arrivals counter)

N_{stable} (Adaptive stability threshold), S_{max} (Maximum advance-time limit)

α, β (Smoothing factors, where $\alpha = 1/8, \beta = 1/4$)

ϵ (Debt clearance threshold)

Phase 1: Delay, Risk and Debt Estimation

$error \leftarrow \Delta t_{arrive} - \mu_{delay}$

$\mu_{delay} \leftarrow \mu_{delay} + \alpha \cdot error$

$dev_{delay} \leftarrow dev_{delay} + \beta(|error| - dev_{delay})$

$Risk_{pred} \leftarrow \max(\Delta t_{arrive}, \mu_{delay}) + dev_{delay}$ ▷ Evaluate tail risk

$Debt_{risk} \leftarrow Debt_{risk} + \alpha \cdot (\max(0, Risk_{pred}) - Debt_{risk})$ ▷ Accumulate or decay debt

$IsDebtFree \leftarrow (Debt_{risk} < \epsilon)$

Phase 2: Trade-off Actuation (Throughput vs. Latency)

if $\Delta t_{arrive} > 0 \vee Risk_{pred} > 0$ **then**

    $S_{next} \leftarrow \min(S_{curr} + 1, S_{max})$ ▷ Preserve throughput

    $count_{stable} \leftarrow 0$

else if $IsDebtFree \wedge (count_{stable} \geq N_{stable})$ **then**

    $S_{next} \leftarrow \max(S_{curr} - 1, 1)$ ▷ Optimize latency

    $count_{stable} \leftarrow 0$

else

    $S_{next} \leftarrow S_{curr}$ ▷ Maintain current timing

    if $Risk_{pred} \leq 0$ **then**

        $count_{stable} \leftarrow count_{stable} + 1$

    end if

end if

return S_{next}

Chapter 4 Experimental Results

This study conducts experiments in an OpenAirInterface (OAI) nFAPI end-to-end environment to verify the proposed delay management method. To ensure the statistical reliability of the evaluation, each throughput value reported represents the average of 180 data points (3 independent trials $\times$ 60 samples per trial).

To validate the hypothesis and the effectiveness of the proposed dynamic trigger scheduling algorithm, the experimental scenarios are arranged as follows: Section 4.1 outlines the hardware and software environment as well as the automated evaluation framework. Section 4.2 validates the Exponentially Weighted Moving Average (EWMA) mechanism for the PNF arrival offset (Δt_{arrive}). Section 4.3 benchmarks the performance of peak throughput and MAC layer Round-Trip Time (RTT) against static baselines. Section 4.4 evaluates the stability of the proposed method under varying degrees of network congestion.

4.1 Experimental Scenario

The testbed environment connects a series of high-performance servers, radio units, and commercial smartphones. The detailed hardware and software configuration of the end-to-end network deployment is shown in the table below.

To ensure consistent data quality for automated measurement and analysis, this study developed an automated evaluation rApp running on the

Table 4.1: Testbed Node Configuration

Unit	**Specification**
Core Network (CN)	Open5GS [20]
Software Protocol Stack / Branch	OpenAirInterface [21] (`2026.w13`)
VNF Server	Intel® Xeon® Gold 6226R [22] with NetXtreme BCM5719 NIC [23,24]
PNF Server	Intel® Xeon® Gold 6433N [25] with Intel I210 NIC [24,26]
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
Commercial Off-The-Shelf (COTS) UE	Samsung Galaxy S20 [27]

Table 4.2: Wireless System Parameters

Parameter	**Setting**
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul Interface	O-RAN F release

Non-Real-Time RAN Intelligent Controller (Non-RT RIC) within the O-RAN Service Management and Orchestration (SMO) framework [28,29], as shown in Figure 4.1. This rApp automatically starts the VNF and PNF. Once the gNB is successfully established, the rApp automatically connects to the control PC via SSH, toggles the UE's airplane mode using the Android Debug Bridge (ADB) [30], and then launches an iPerf [31] server on the Core Network server via SSH. Subsequently, it automatically executes a series of predefined iPerf test cases. After that, the rApp collects all log files via O1 SFTP and uses Python scripts to automatically plot charts for subsequent data analysis. During the experiments, we also used tcpdump [32] for packet sniffing, and utilized Linux PTP [33] and TSN time synchronization mechanisms [34] to ensure baseline timing among network nodes, while underlying data transmission relied on DPDK [35] for packet acceleration. A key capability of this framework is the ability to automate network delay simulations on the MAC feedback link ($T_5 - T_4$) and fronthaul link ($T_2 - T_1$) using Linux Traffic Control (TC), which is crucial for verifying the stability of the disaggregated architecture under non-deterministic latency.

4.2 Scenario I: EWMA Validation for PNF Arrival Offset (Δt_{arrive})

This scenario verifies the effectiveness of applying the Exponentially Weighted Moving Average (EWMA) to smooth the PNF arrival offset (Δt_{arrive}). Due to network jitter and processing delays, the raw timing offset received from the PNF can exhibit high variability. Figure 4.2 illustrates the impact

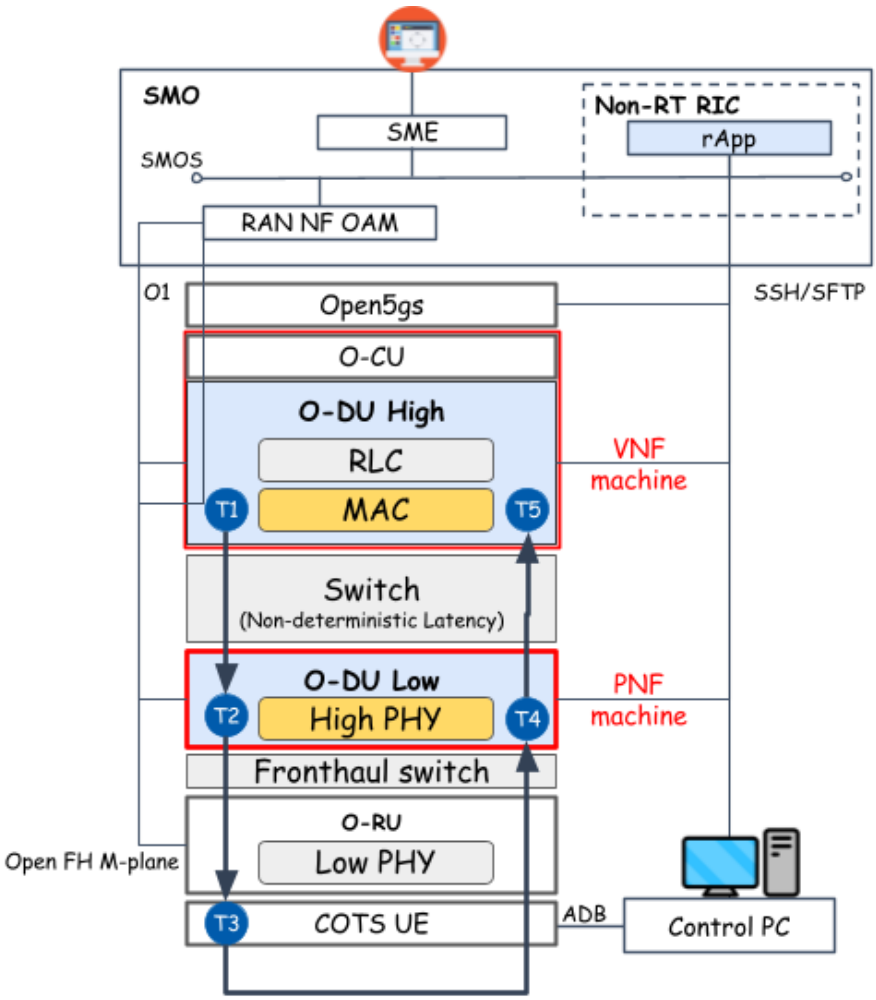
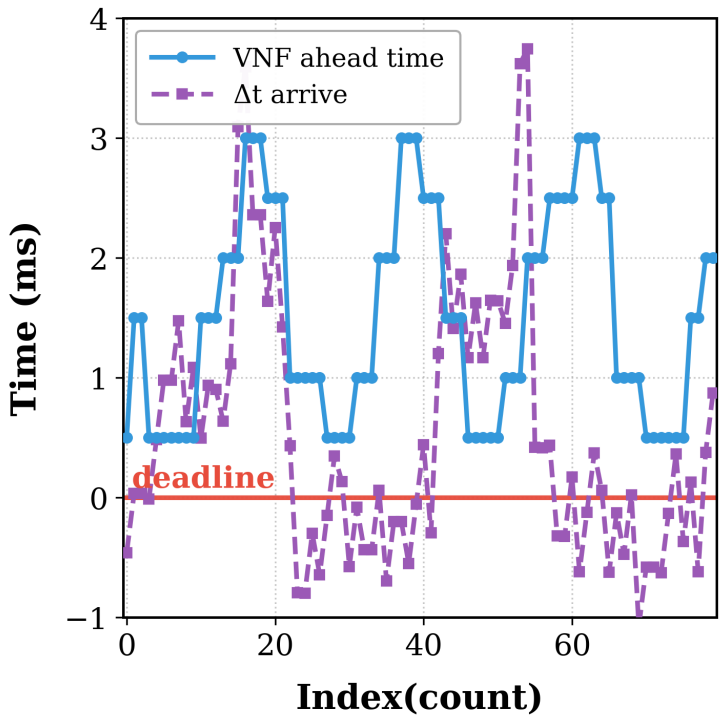
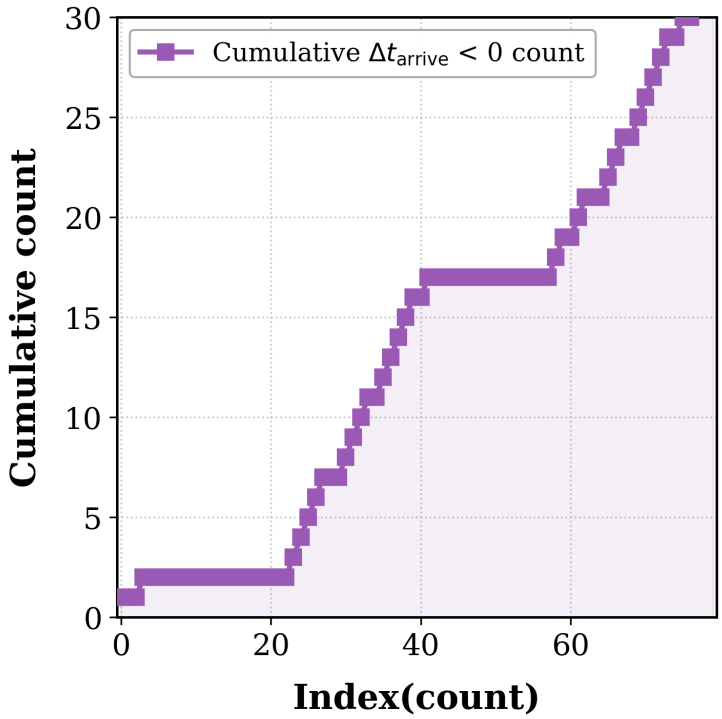


Figure 4.1: Architecture of the rApp automated evaluation framework.

of EWMA processing. Without EWMA (Figure 4.2a), the arrival offset fluctuates drastically, which can lead to unstable scheduling decisions. By applying EWMA (Figure 4.2b), high-frequency jitter is effectively filtered, providing a stable baseline for the timing adjustment algorithm. Furthermore, as shown in Figures 4.2c and 4.2d, the smoothed data allows the system to make relatively stable judgments. When Δt_{arrive} begins to show a downward trend, the algorithm steadily identifies this trend and actively advances more time (S_{ahead}) to maintain synchronization stability.

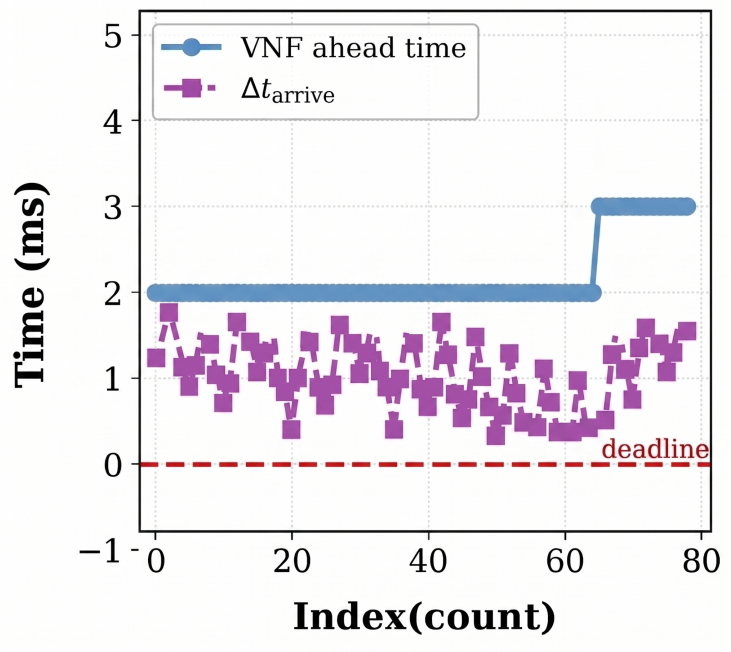


(a) Without EWMA

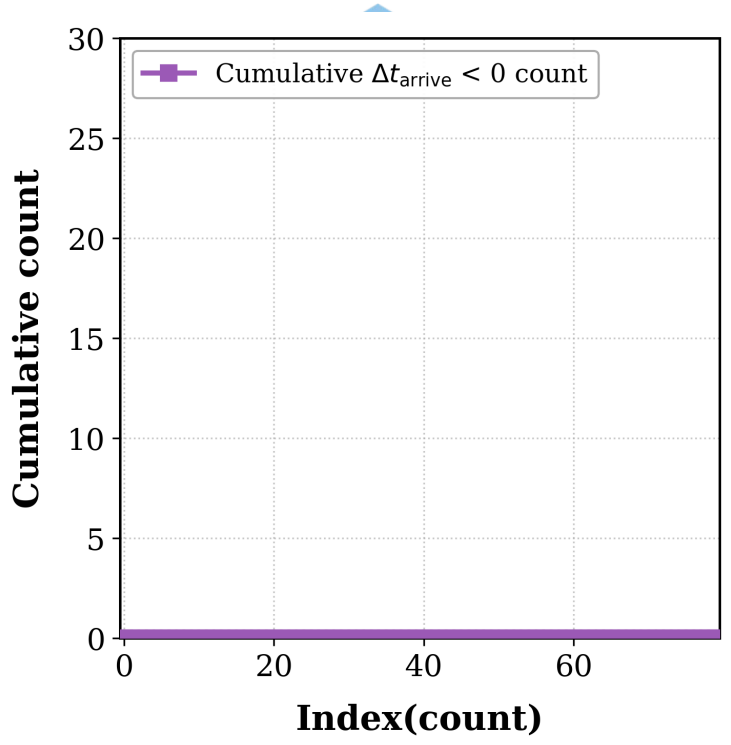


(b) With EWMA

Figure 4.2: EWMA processing validation and active adjustment for PNF arrival offset (Δt_{arrive}).



(c) Trend Observation



(d) Active Advance Adjustment

Figure 4.2: EWMA processing validation and active adjustment for PNF arrival offset (Δt_{arrive}) (cont.).

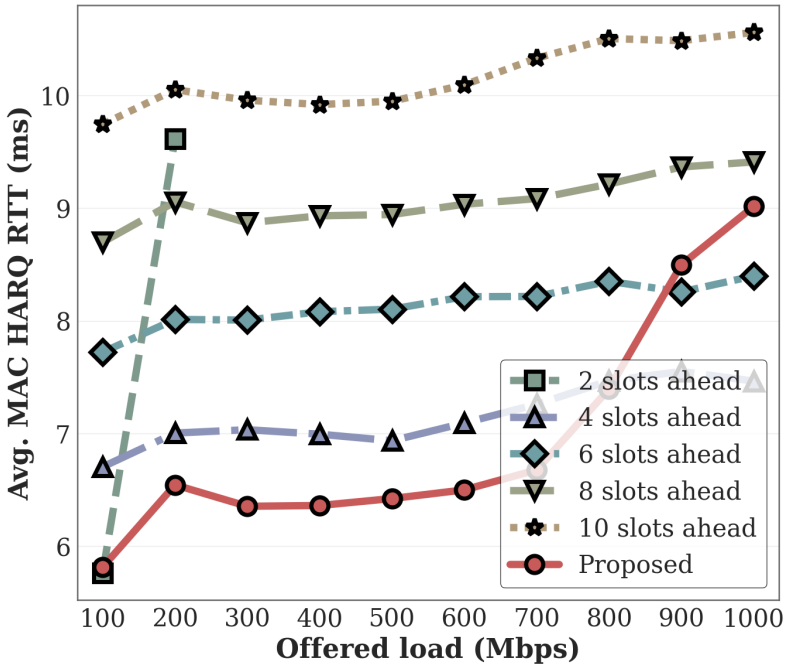
4.3 Scenario II: Performance Benchmarking

In this scenario, we evaluate the benchmark performance of the proposed adaptive timing mechanism against a standard static slots ahead configuration. Figure 4.3 presents the performance evaluation in terms of MAC layer latency and system throughput.

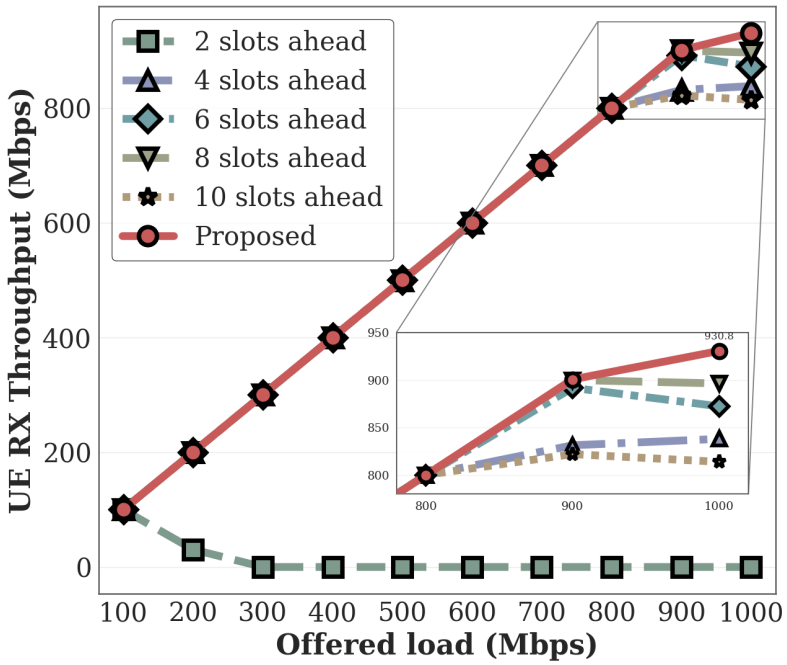
Figure 4.3 displays the performance evaluation under varying Offered Load. The proposed algorithm prioritizes ensuring optimal throughput, consistently maintaining optimal throughput performance under all load conditions. Simultaneously, while ensuring this optimal throughput, the algorithm maximizes the selection of the optimal slots ahead to optimize the MAC layer HARQ Round-Trip Time (RTT), effectively avoiding unnecessary extra overhead.

4.4 Scenario III: Stability under Network Congestion

To test the system's robustness, we utilized Linux TC to inject artificial network congestion into the $T_2 - T_1$ and $T_5 - T_4$ segments, altering the One-Way Delay (OWD) and jitter. As shown in Figure 4.4, when the delay exceeds the fixed timing window, the static configuration experiences a severe drop in throughput and frequent connection resets. In contrast, the proposed adaptive timing control dynamically expands the slots ahead (S_{ahead}). This expansion allows the MAC scheduler to trigger messages in advance, ensuring that messages can still arrive at the PNF within the valid



(a) MAC Latency and Reliability



(b) Throughput Optimization

Figure 4.3: Performance Evaluation: MAC RTT Efficiency and System Throughput.

timing window despite the increased transmission delay. Consequently, even at congestion levels that would previously cause complete synchronization failure, the system can maintain stable throughput.

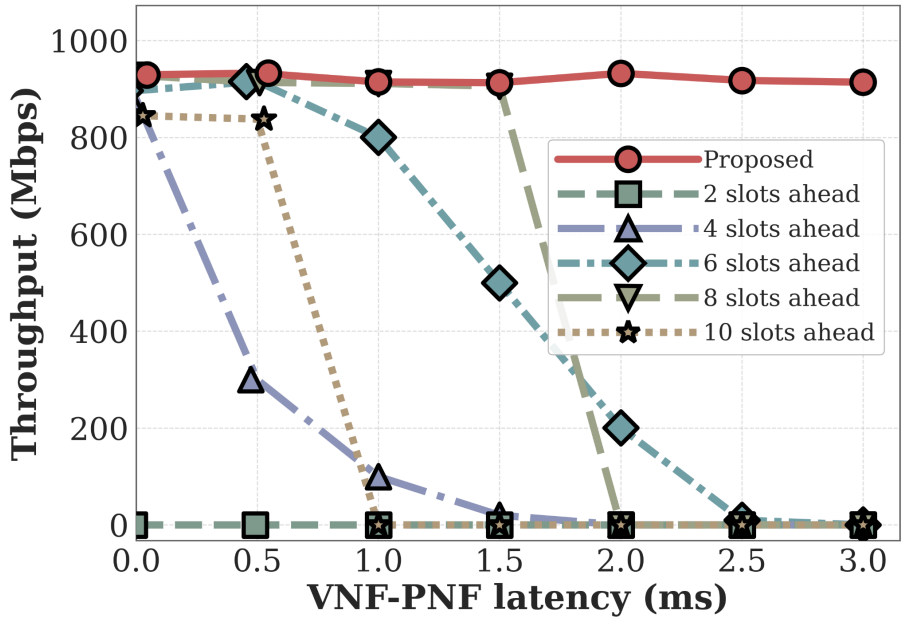


Figure 4.4: System stability and throughput performance under varying degrees of network congestion.

Chapter 5 Conclusion

This study addresses the transmission latency issues in the O-RAN Open Radio Access Network architecture by proposing an Adaptive Slots Ahead mechanism, with a specific focus on the Option 6 functional split architecture utilizing the nFAPI interface. In a non-ideal packet-switched network environment, traditional static slots ahead configurations cannot effectively handle unpredictable network jitter and server processing delays, easily leading to scheduling timeouts and synchronization failures. To overcome this challenge, this study designs a dynamic delay management algorithm based on the Exponentially Weighted Moving Average (EWMA) model. Through closed-loop feedback control, it continuously tracks and predicts end-to-end network transmission latency characteristics.

By integrating the OpenAirInterface (OAI) open-source software with commercial Pegatron O-RU equipment into an end-to-end physical verification platform, the experimental results demonstrate that the proposed delay management architecture significantly improves system stability. Under simulated network congestion scenarios, this mechanism actively compensates for timing offsets, ensuring that MAC layer scheduling commands accurately fall within the timing window of the physical layer, thereby completely eliminating connection drops caused by missing slot packets. Compared to traditional solutions that rely on expensive hardware acceleration or strict hardware-level PTP synchronization, the software-based solution proposed in this study demonstrates extremely high deployment flexibility and cost-effectiveness on general-purpose Commercial Off-The-Shelf (COTS) servers, providing a highly reliable key technological foundation

for the commercialization of future 5G/6G heterogeneous networks.

References

[1] O-RAN Alliance, "O-RAN Specifications." [Online]. Available: `https://www.o-ran.org/specifications`, 2025.

[2] 3GPP, "Study on new radio access technology: Radio access architecture and interfaces," Tech. Rep. TR 38.801, 3rd Generation Partnership Project (3GPP), 2017. Version 14.0.0.

[3] B. Agarwal, R. Irmer, D. Lister, and G.-M. Muntean, "Open ran for 6g networks: Architecture, use cases and open issues," *IEEE Communications Surveys & Tutorials*, vol. 28, no. 3, pp. 2881–2924, 2025.

[4] L. M. P. Larsen, A. Checko, and H. L. Christiansen, "A survey of the functional splits in 5g next-generation radio access networks," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2349–2372, 2019.

[5] J. Pérez-Romero, O. Sallent, A. Gelonch, X. Gelabert, B. Klaiqi, M. Kahn, and D. Campoy, "A tutorial on the characterisation and modelling of low layer functional splits for flexible radio access networks in 5g and beyond," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2791–2833, 2023.

[6] X. Foukas *et al.*, "Computational resource consumption of 5g virtualized ran," in *2021 IEEE International Conference on Communications (ICC)*, IEEE, 2021.

[7] O-RAN Alliance, "O-ran fronthaul working group control, user and synchronization plane specification," Tech. Rep. O-RAN.WG4.CUS.0-v11.00, O-RAN Alliance, 2023.

[8] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, "X5g: An open, programmable, multi-vendor, end-to-end, private 5g o-ran testbed with nvidia arc and openairinterface," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11305–11322, 2025.

[9] B. Khan, N. Nidhi, R. R. Paulo, A. Mihovska, and F. J. Velez, "Cost revenue trade-off for the 5g nr small cell network in the sub-6 ghz operating band," in *2022 25th International Symposium on Wireless Personal Multimedia Communications (WPMC)*, pp. 64–69, 2022.

[10] S. C. Forum, "5g fapi: Phy api specification," Tech. Rep. SCF222, Small Cell Forum, 2021.

[11] S. C. Forum, "5g network fapi (nfapi) specification," Tech. Rep. SCF225, Small Cell Forum, 2020.

[12] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 32–40, 2018.

[13] Small Cell Forum (SCF), "nfapi and fapi specification," Technical Specification SCF082, Small Cell Forum (SCF), 2021.

[14] S.-Q. Lee and M.-S. Lee, "A method of adaptive low-latency downlink transmission on fapi based 5g nr gnb," in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*, IEEE, 2022.

[15] A. S. Mohammed, K. S. Tharakan, H. A. Ammar, H. ElSawy, and H. S. Hassanein, "Fronthaul network planning for hierarchical and radio-stripes-enabled cf-mmimo in o-ran," *IEEE Transactions on Wireless Communications*, vol. 25, pp. 14781–14796, 2026.

[16] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, A. Garcia-Saavedra, and X. Costa-Pérez, "Oranus: Latency-tailored orchestration via stochastic network calculus in 6g o-ran," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 61–70, 2024.

[17] Y. Chen, Y. T. Hou, W. Lou, J. H. Reed, and S. Kompella, "M3: A sub-millisecond scheduler for multi-cell mimo networks under c-ran architecture," in *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*, 2022.

[18] J. Lv, Y. Gao, Z. Ding, Y. Lin, X. You, G. Yang, and W. Dong, "Providing ue-level qos support by joint scheduling and orchestration for 5g vran," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 51–60, 2024.

[19] X. Wang *et al.*, "An open-source implementation of the 5g nr functional split option 6," in *2022 IEEE International Conference on Communications (ICC)*, pp. 1–6, 2022.

[20] "Open5gs: Open source 5g core network." [Online]. Available: `https://open5gs.org/`, 2024.

[21] OpenAirInterface, "OpenAirInterface Official Website." [Online]. Available: `https://openairinterface.org/`, 2025.

[22] "Intel xeon gold 6226r processor (22m cache, 2.90 ghz)." [Online]. Available: `https://www.intel.com/content/www/us/en/products/sku/199347/intel-xeon-gold-6226r-processor-22m-cache-2-90-ghz/specifications.html`, 2025.

[23] "Broadcom netxtreme bcm5719 quad-port 1gbe base-t adapter." [Online]. Available: `https://www.dell.com/zh-tw/shop/broadcom-5719-%E5%9B%9B%E9%80%A3%E6%8E%A5%E5%9F%A0-1gbe-base-t-%E9%85%8D%E6%8E%A5%E5%8D%A1-pcie-%E5%85%A8%E9%AB%98/apd/540-bdrj/%E7%B6%B2%E8%B7%AF%E8%A8%AD%E5%82%99`, 2026.

[24] "Network interface controller." [Online]. Available: `https://en.wikipedia.org/wiki/Network_interface_controller`, 2024.

[25] "Intel xeon gold 6433n processor (42m cache, 2.00 ghz)." [Online]. Available: `https://www.intel.com/content/www/us/en/products/sku/232148/intel-xeon-gold-6433n-processor-42m-cache-2-00-ghz/specifications.html`, 2025.

[26] “Intel ethernet server adapter i210-t1.” [Online]. Available: https://www.intel.com.tw/content/www/tw/zh/products/sku/68668/intel-ethernet-server-adapter-i210t1/specifications.html, 2026.

[27] 3GPP, “User equipment (ue) radio transmission and reception,” Technical Specification (TS) 38.101, 3rd Generation Partnership Project (3GPP), Sept. 2024. Version 16.17.0.

[28] O-RAN Working Group 1, *O-RAN Architecture Description*. O-RAN Alliance, June 2025. Rev. 14.00.

[29] O-RAN Working Group 2, *Non-RT RIC Architecture*. O-RAN Alliance, February 2022. O-RAN.WG2.Non-RT-RIC-ARCH-v07.00.

[30] “Android debug bridge (adb) command-line tool.” [Online]. Available: https://developer.android.com/tools/adb, 2024.

[31] E. E. S. Network), “iperf3 - the ultimate speed test tool for tcp, udp and sctp.” [Online]. Available: https://iperf.fr/, 2025.

[32] The Tcpdump Group, “tcpdump.” [Online]. Available: https://www.tcpdump.org/, 2025.

[33] R. Cochran, “Linux ptp v3.1.1: Precision time protocol (ptp) implementation for linux.” [Online]. Available: https://github.com/richardcochran/linuxptp/releases/tag/v3.1.1, 2023.

[34] TSN Community, “Time synchronization - time sensitive networking (tsn).” [Online]. Available: https://tsn.readthedocs.io/timesync.html#checking-clocks-synchronization, 2024.

[35] DPDK Project, “Data plane development kit (dpdk) v20.11.9.” [Online]. Available: https://doc.dpdk.org/guides-20.11/, 2023. Long Term Support (LTS) Release.